

学生姓名	戴仟仟	专业班级	电气工程及其自动化 9 班	学 号	20240682	
课 程 名 称	数字电子技术实验 (II)	实验项目 名 称	FPGA 电子秒表设计	实验项目类型		
				验证	演示	综合
指 导 教 师	汪 金 刚	成 绩		√		

一、实验目的

1. 进一步学习由 FPGA 构成的时序逻辑电路设计方法；
2. 学习利用 Verilog 语言实现时钟分频模块、按键消抖模块、计数控制模块和译码显示模块等；
3. 学习电子秒表的工作原理和基于 FPGA 的小型电路系统设计方法。

二、实验原理（及其相关定义）

1. 电子秒表是生产生活中常用的一种计时设备，它具有操作简单、计时精度和准确性高、以及使用寿命长等特点，其广泛应用于科学研究、体育运动及国防等领域。电子秒表的基本工作原理是在特定频率的连续时钟脉冲触发下，计数器按一定时间制式循环计数，并译码显示出时间数值。

2. 本实验基于 FPGA 完成电子秒表的设计，其基本构成模块包括时钟分频模块、按键消抖模块、计数控制模块和译码显示模块，该系统框图如图 7-1 所示：

3. 电子秒表的四大模块

(1) 时钟分频模块

系统板载时钟是 100MHz，但各模块的工作时钟不同，因此需要对系统时钟进行分频或倍频，时钟分频或倍频可以采用计数器方式或时钟 IP 核来完成。按键消抖模块中，由于按键按下后需经过 5-20ms 才能达到稳定状态，因此该模块的时钟频率设定为 50Hz 比较合适。计数控制模块中，可以根据不同的时间精度确定工作时钟，如果时间精度设置为 10ms，则该模块的时钟频率可设定为 100Hz。译码显示模块中，数码管采用动态显示方式，每位数码管的点亮时间为 1-2ms，因此该模块的时钟频率设定为 1000Hz 比较合适。

(2) 按键消抖模块

实验板上所用按键开关为机械弹性开关，当机械触点闭合、断开时，由于机械触点的弹性作用，一个按键开关在闭合时不会马上稳定地接通，在断开时也不会一下子断开，需要 5-20ms 才能达到稳定。因而，在闭合和断开的瞬间均伴随有一连串的抖动，为了不产生这种抖动现象所做的措施就是按键消抖。

(3) 计数控制模块

计数控制模块主要完成计数、进制转换和通过按键实现电子秒表的启动、停止、清零等控制功能。计数器在时钟信号的驱动下，百分之一秒寄存器以 10ms 为单位进行计数。当百分之一秒寄存器计时达到 10 时，十分之一秒寄存器加一。当十分之一秒寄存器计时达到 10 时，秒寄存器加一。当秒寄存器计时达到 60 时，分寄存器加一。当分寄存器计时达到 60 时，清零。

(4) 译码显示模块

数码管驱动方式分成静态驱动和动态扫描驱动，静态驱动虽然操作方便，但占用的 I/O 口较多。当需要用到多位数码管时，为了减少数码管占用的 I/O 口，可将其段选（数码管的 a、b、c 等引脚）连接在一起，而位选（数码管的公共端）独立控制，采用动态扫描进行驱动。以两位数码管为例，其内部连接如图 7-3。

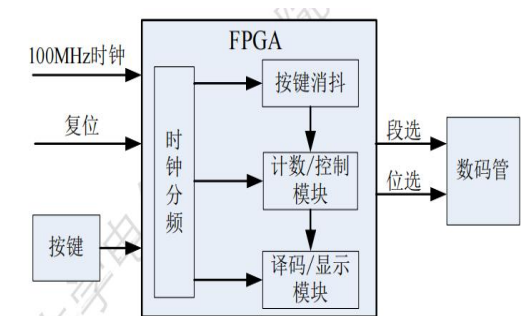


图 7-1 电子秒表系统框图

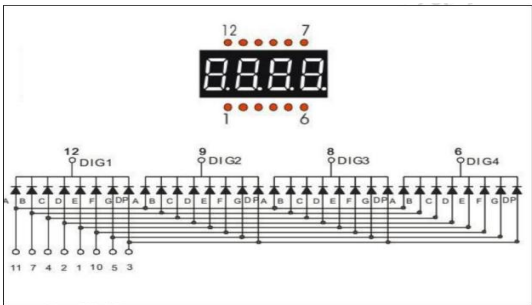


图 7-3 四位共阴数码管引脚图

三、使用仪器、材料数量及其用途

1. 电脑一台 用 vivado 软件设计电子秒表，并完成仿真
2. FPGA 实验板一套 进行电子秒表的板级验证

四、实验步骤（含简略步骤、电路图）

1. 实验基本要求

- （1）设计一个电子秒表，该秒表能在 0 秒~59 分 59 秒 99 厘秒范围进行计时。采用 6 个数码管从左至右依次显示“十分位、分位、十秒位、秒位、分秒位、厘秒位”；
- （2）数码管显示格式为“00.00.00”，即用小数点将分、秒、厘秒隔开；
- （3）计时精度达到 10ms；
- （4）设计复位开关，复位开关可以在任何情况下使用，使用以后计时器清零，并做好下一次计时的准备。
- （5）设计启动（继续）/停止开关，采用一个按键来切换秒表的启动（继续）和停止状态。
- （6）设计一分钟倒计时功能，采用一个按键控制倒计时的启动，当按键被按下，可立即进入倒计时功能，开始一分钟倒计时。倒计时时间到，计时停止，同时可亮起一个 LED 提示。复用复位按键，可跳出倒计时功能，重新回到秒表状态，倒计时提示 LED 熄灭。

五、实验过程原始记录(数据、图表、仿真、计算等)

1. 源程序代码：

时钟分频模块

```
module clk_div(  
    input          clk_100m,  
    input          rst_n,  
    output reg     clk_50hz,  
    output reg     clk_100hz,  
    output reg     clk_1khz  
);  
reg [19:0] cnt_50hz;  
always @(posedge clk_100m or negedge rst_n) begin  
    if(!rst_n) begin  
        cnt_50hz <= 20'd0;  
        clk_50hz <= 1'b0;  
    end else if(cnt_50hz == 20'd999_999) begin  
        cnt_50hz <= 20'd0;  
        clk_50hz <= ~clk_50hz;  
    end else begin  
        cnt_50hz <= cnt_50hz + 1'b1;  
    end  
end  
reg [18:0] cnt_100hz;  
always @(posedge clk_100m or negedge rst_n) begin  
    if(!rst_n) begin  
        cnt_100hz <= 19'd0;  
        clk_100hz <= 1'b0;  
    end else if(cnt_100hz == 19'd499_999) begin  
        cnt_100hz <= 19'd0;  
        clk_100hz <= ~clk_100hz;  
    end else begin  
        cnt_100hz <= cnt_100hz + 1'b1;  
    end  
end
```

```

        end
    end
    reg [16:0] cnt_1khz;
    always @(posedge clk_100m or negedge rst_n) begin
        if(!rst_n) begin
            cnt_1khz <= 17'd0;
            clk_1khz <= 1'b0;
        end else if(cnt_1khz == 17'd49_999) begin
            cnt_1khz <= 17'd0;
            clk_1khz <= ~clk_1khz;
        end else begin
            cnt_1khz <= cnt_1khz + 1'b1;
        end
    end
end
endmodule

```

消抖模块

```

module key_debounce(
    input        clk_50hz,
    input        rst_n,
    input        btn_in,
    output       btn_out
);
    reg btn0;
    reg btn1;
    reg btn2;
    assign btn_out = btn0 & btn1 & btn2;
    always @(posedge clk_50hz or negedge rst_n)
    begin
        if(!rst_n)
        begin
            btn0 <= 1'b0;
            btn1 <= 1'b0;
            btn2 <= 1'b0;
        end
        else
        begin
            btn0 <= btn_in;
            btn1 <= btn0;
            btn2 <= btn1;
        end
    end
end
endmodule

```

计数控制模块

```

module count_ctrl(
    input        clk_100hz,
    input        rst_n,
    input        key_start,
    input        key_countd,

```

```

        output [23:0] disp_time,
        output reg    led_alarm
    );
    reg [3:0] msec_l, msec_h, sec_l, sec_h, min_l, min_h;
    reg      cnt_en;
    reg      countd_en;
    reg [15:0] countd_val;
    reg      key_start_r;
    reg      key_countd_r;
    always @(posedge clk_100hz or negedge rst_n) begin
        if(!rst_n) begin
            key_start_r <= 1'b0;
            key_countd_r <= 1'b0;
        end else begin
            key_start_r <= key_start;
            key_countd_r <= key_countd;
        end
    end
    wire key_start_pulse = key_start & ~key_start_r;
    wire key_countd_pulse = key_countd & ~key_countd_r;
    always @(posedge clk_100hz or negedge rst_n) begin
        if(!rst_n) begin
            cnt_en <= 1'b0;
        end else if(key_start_pulse) begin
            cnt_en <= ~cnt_en;
        end
    end
    always @(posedge clk_100hz or negedge rst_n) begin
        if(!rst_n) begin
            countd_en <= 1'b0;
            countd_val <= 16'd6000;
            led_alarm <= 1'b0;
        end else if(key_countd_pulse) begin
            countd_en <= 1'b1;
            countd_val <= 16'd6000;
            led_alarm <= 1'b0;
        end else if(countd_en && (countd_val == 16'd0)) begin
            led_alarm <= 1'b1;
            countd_en <= 1'b0;
        end
    end
    always @(posedge clk_100hz or negedge rst_n) begin
        if(!rst_n) begin
            msec_l <= 4'd0; msec_h <= 4'd0;
            sec_l  <= 4'd0; sec_h  <= 4'd0;
            min_l  <= 4'd0; min_h  <= 4'd0;
        end else if(countd_en) begin
            if(countd_val > 16'd0) begin

```

```

countd_val <= countd_val - 1'b1;
min_h <= ((countd_val - 1) / 6000) / 10;
min_l <= ((countd_val - 1) / 6000) % 10;
sec_h <= (((countd_val - 1) % 6000) / 100) / 10;
sec_l <= (((countd_val - 1) % 6000) / 100) % 10;
msec_h <= (((countd_val - 1) % 100) / 10);
msec_l <= ((countd_val - 1) % 10);
end
end else if(cnt_en) begin
    msec_l <= msec_l + 1'b1;
    if(msec_l == 4'd9) begin
        msec_l <= 4'd0;
        msec_h <= msec_h + 1'b1;
        if(msec_h == 4'd9) begin
            msec_h <= 4'd0;
            sec_l <= sec_l + 1'b1;
            if(sec_l == 4'd9) begin
                sec_l <= 4'd0;
                sec_h <= sec_h + 1'b1;
                if(sec_h == 4'd5) begin
                    sec_h <= 4'd0;
                    min_l <= min_l + 1'b1;
                    if(min_l == 4'd9) begin
                        min_l <= 4'd0;
                        min_h <= min_h + 1'b1;
                        if(min_h == 4'd5) begin
                            min_h <= 4'd0;
                        end
                    end
                end
            end
        end
    end
end
end
end
end
end
end
end
assign disp_time = {min_h, min_l, sec_h, sec_l, msec_h, msec_l};
endmodule

译码显示模块
module smg(
    input clk_1khz,
    input rst_n,
    input [23:0] dispdata,
    output reg [7:0] seg1,
    output reg [7:0] seg2,
    output reg [5:0] an
);
reg [3:0] disp_dat;
reg [2:0] disp_bit;
```

```

always @(posedge clk_1khz or negedge rst_n)
begin
    if(!rst_n)
    begin
        an <= 6'b111111;
        disp_bit <= 3'd0;
        disp_dat <= 4'd0;
    end else begin
        disp_bit <= (disp_bit == 3'd5) ? 3'd0 : disp_bit + 1'b1;
        case(disp_bit)
        3'd0: begin
            disp_dat <= dispdata[23:20];
            an <= 6'b111110;
        end
        3'd1: begin
            disp_dat <= dispdata[19:16];
            an <= 6'b111101;
        end
        3'd2: begin
            disp_dat <= dispdata[15:12];
            an <= 6'b111011;
        end
        3'd3: begin
            disp_dat <= dispdata[11:8];
            an <= 6'b110111;
        end
        3'd4: begin
            disp_dat <= dispdata[7:4];
            an <= 6'b101111;
        end
        3'd5: begin
            disp_dat <= dispdata[3:0];
            an <= 6'b011111;
        end
        default: begin
            disp_dat <= 4'd0;
            an <= 6'b111111;
        end
    endcase
end
end

always@(*) begin
    seg1 = 8'b11111111;
    seg2 = 8'b11111111;
    case(an)
        6'b111110, 6'b101111: begin
            case(disp_dat)

```

```

4'h0: seg1 = 8'b11111100;
4'h1: seg1 = 8'b01100000;
4'h2: seg1 = 8'b11011010;
4'h3: seg1 = 8'b11110010;
4'h4: seg1 = 8'b01100110;
4'h5: seg1 = 8'b10110110;
4'h6: seg1 = 8'b00111110;
4'h7: seg1 = 8'b11100000;
4'h8: seg1 = 8'b11111110;
4'h9: seg1 = 8'b11100110;
default: seg1 = 8'b11111111;
endcase
end
6'b111101, 6'b111011: begin
    case(dis_dat)
4'h0: seg2 = 8'b11111100;
4'h1: seg2 = 8'b01100000;
4'h2: seg2 = 8'b11011010;
4'h3: seg2 = 8'b11110010;
4'h4: seg2 = 8'b01100110;
4'h5: seg2 = 8'b10110110;
4'h6: seg2 = 8'b00111110;
4'h7: seg2 = 8'b11100000;
4'h8: seg2 = 8'b11111110;
4'h9: seg2 = 8'b11100110;
default: seg2 = 8'b11111111;
    endcase
end
6'b110111, 6'b011111: begin
    case(dis_dat)
4'h0: seg1 = 8'b11111101;
4'h1: seg1 = 8'b01100001;
4'h2: seg1 = 8'b11011011;
4'h3: seg1 = 8'b11110011;
4'h4: seg1 = 8'b01100111;
4'h5: seg1 = 8'b10110111;
4'h6: seg1 = 8'b00111111;
4'h7: seg1 = 8'b11100001;
4'h8: seg1 = 8'b11111111;
4'h9: seg1 = 8'b11100111;
default: seg1 = 8'b11111111;
    endcase
end
default: begin
    seg1 = 8'b11111111;
    seg2 = 8'b11111111;
end
endcase

```

```
end
endmodule
顶层文件
module top_stopwatch(
    input          clk_100m,
    input          rst,
    input          key_start,
    input          key_countd,
    output [7:0]   seg1,
    output [7:0]   seg2,
    output [5:0]   an,
    output         led_alarm
);
wire clk_50hz;
wire clk_100hz;
wire clk_1khz;
wire [23:0] disp_time;
wire key_start_deb;
wire key_countd_deb;
wire rst_n = ~rst;
clk_div u_clk_div(
    .clk_100m(clk_100m),
    .rst_n(rst_n),
    .clk_50hz(clk_50hz),
    .clk_100hz(clk_100hz),
    .clk_1khz(clk_1khz)
);
key_debounce u_key_start(
    .clk_50hz(clk_50hz),
    .rst_n(rst_n),
    .btn_in(key_start),
    .btn_out(key_start_deb)
);
key_debounce u_key_countd(
    .clk_50hz(clk_50hz),
    .rst_n(rst_n),
    .btn_in(key_countd),
    .btn_out(key_countd_deb)
);
count_ctrl u_count_ctrl(
    .clk_100hz(clk_100hz),
    .rst_n(rst_n),
    .key_start(key_start_deb),
    .key_countd(key_countd_deb),
    .disp_time(disp_time),
    .led_alarm(led_alarm)
);
smg u_smg(
```



```

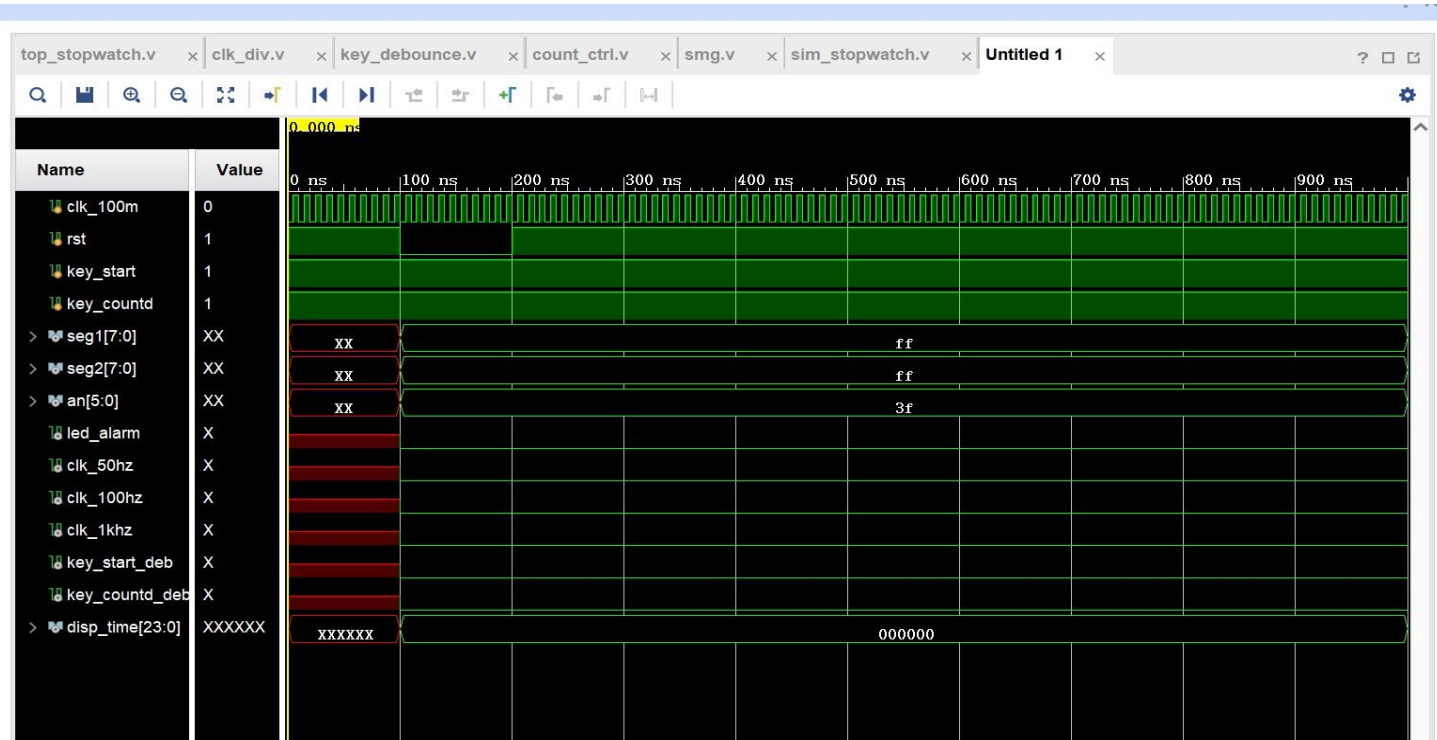
        .clk_1khz(clk_1khz),
        .rst_n(rst_n),
        .dispdata(disp_time),
        .seg1(seg1),
        .seg2(seg2),
        .an(an)
    );
endmodule
仿真文件代码:
module sim_stopwatch;
reg          clk_100m;
reg          rst;
reg          key_start;
reg          key_countd;
wire [7:0]   seg1;
wire [7:0]   seg2;
wire [5:0]   an;
wire         led_alarm;
wire         clk_50hz;
wire         clk_100hz;
wire         clk_1khz;
wire         key_start_deb;
wire         key_countd_deb;
wire [23:0]  disp_time;
top_stopwatch u_top_stopwatch(
    .clk_100m(clk_100m),
    .rst(rst),
    .key_start(key_start),
    .key_countd(key_countd),
    .seg1(seg1),
    .seg2(seg2),
    .an(an),
    .led_alarm(led_alarm)
);
assign clk_50hz    = u_top_stopwatch.clk_50hz;
assign clk_100hz   = u_top_stopwatch.clk_100hz;
assign clk_1khz    = u_top_stopwatch.clk_1khz;
assign key_start_deb = u_top_stopwatch.key_start_deb;
assign key_countd_deb = u_top_stopwatch.key_countd_deb;
assign disp_time    = u_top_stopwatch.disp_time;
initial begin
    clk_100m = 1'b0;
    forever #5 clk_100m = ~clk_100m;
end
initial begin
    rst          = 1'b1;
    key_start    = 1'b1;
    key_countd   = 1'b1;

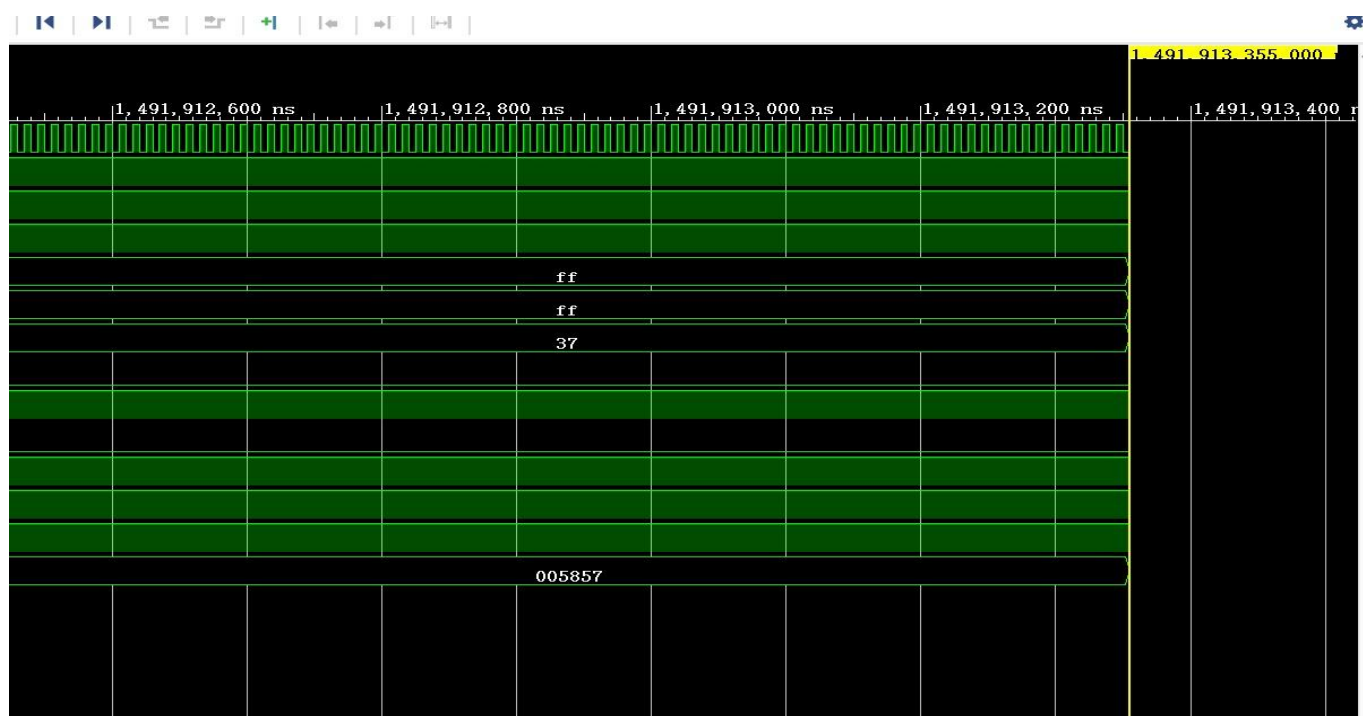
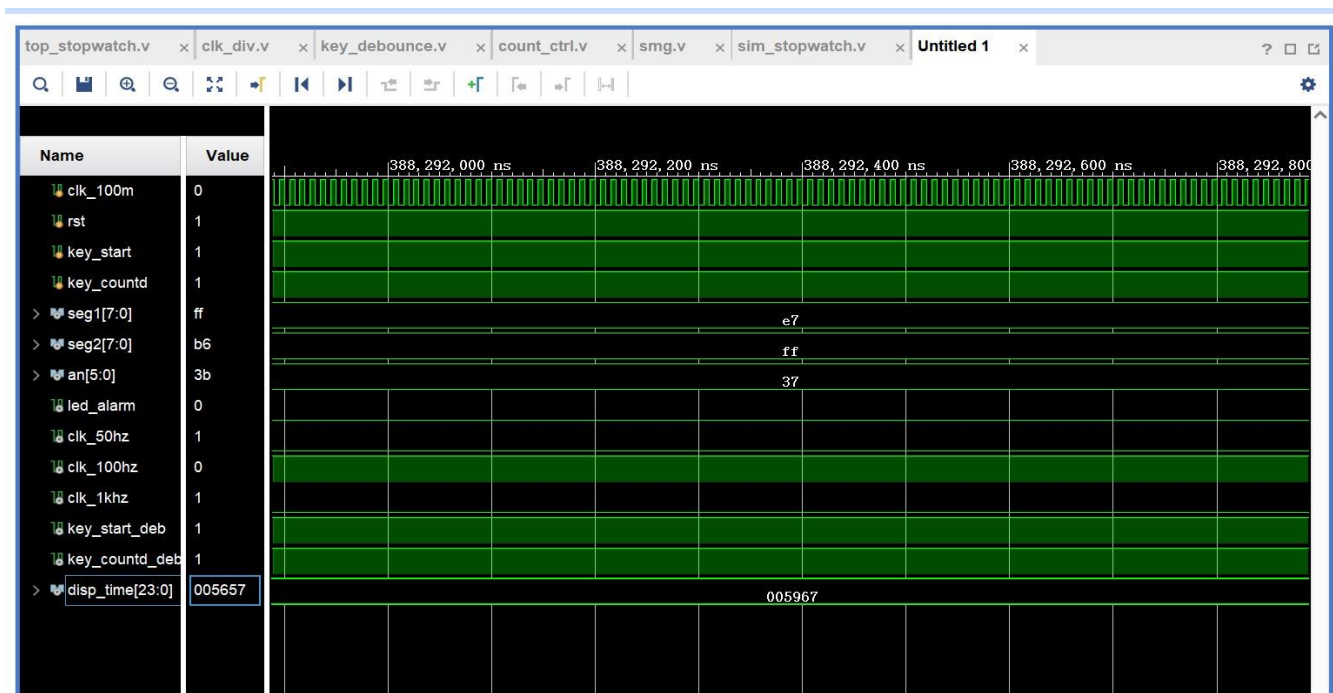
```

```
#100 rst = 1'b0;
#100 rst = 1'b1;
#10000;
key_start = 1'b0;
#20 key_start = 1'b1;
#10 key_start = 1'b0;
#10 key_start = 1'b1;
#15 key_start = 1'b0;
#50000;
key_start = 1'b1;
#50000000;
key_start = 1'b0; #50000; key_start = 1'b1;
#10000000;
key_countd = 1'b0; #50000; key_countd = 1'b1;
#10000000;
#10000000;
$finish;

end
endmodule
```

2. 仿真波形图：





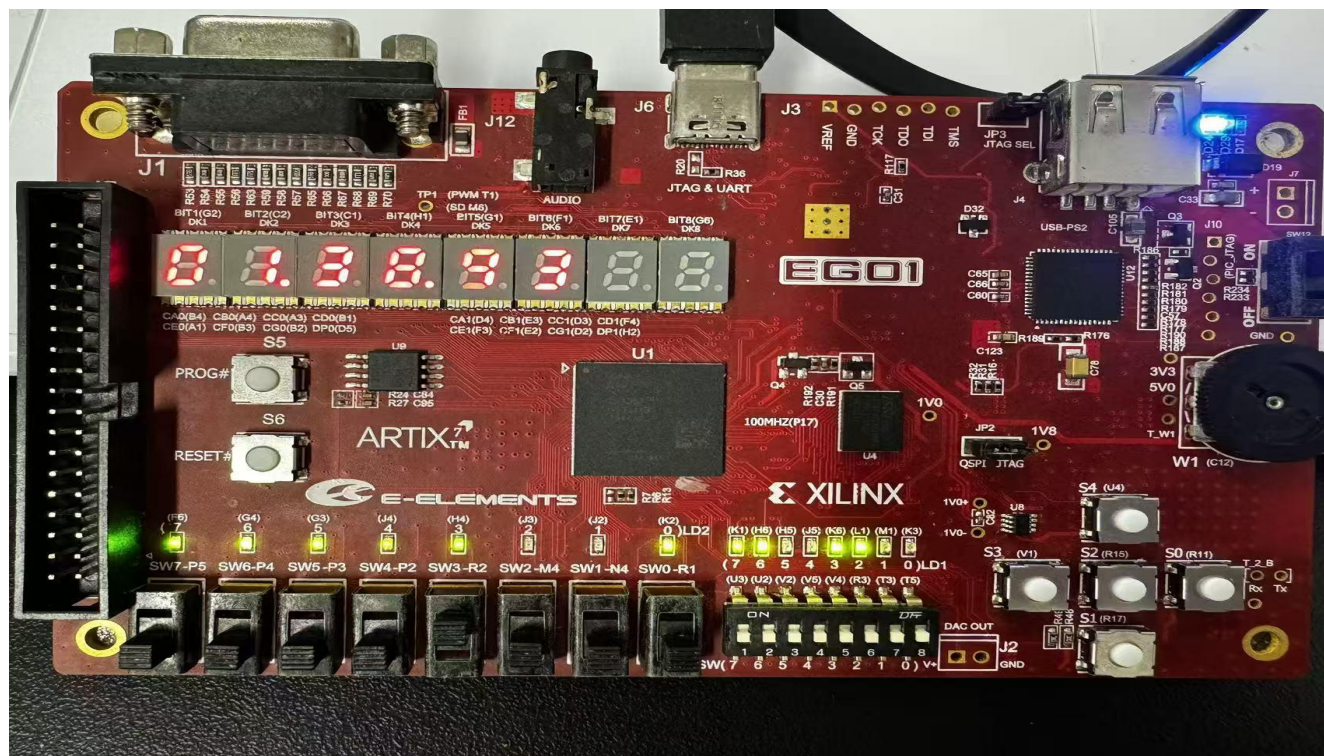
3. 引脚约束文件

```
set_property IOSTANDARD LVCMOS33 [get_ports {an[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
```

set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports key_countd]
set_property IOSTANDARD LVCMOS33 [get_ports clk_100m]
set_property IOSTANDARD LVCMOS33 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports led_alarm]
set_property IOSTANDARD LVCMOS33 [get_ports key_start]
set_property PACKAGE_PIN P17 [get_ports clk_100m]
set_property PACKAGE_PIN P15 [get_ports rst]
set_property PACKAGE_PIN R11 [get_ports key_start]
set_property PACKAGE_PIN R17 [get_ports key_countd]
set_property PACKAGE_PIN K3 [get_ports led_alarm]
set_property PACKAGE_PIN B4 [get_ports {seg[0]}]
set_property PACKAGE_PIN A4 [get_ports {seg[1]}]
set_property PACKAGE_PIN A3 [get_ports {seg[2]}]
set_property PACKAGE_PIN B1 [get_ports {seg[3]}]
set_property PACKAGE_PIN A1 [get_ports {seg[4]}]
set_property PACKAGE_PIN B3 [get_ports {seg[5]}]
set_property PACKAGE_PIN B2 [get_ports {seg[6]}]
set_property PACKAGE_PIN D5 [get_ports {seg[7]}]
set_property PACKAGE_PIN G2 [get_ports {an[0]}]
set_property PACKAGE_PIN C2 [get_ports {an[1]}]
set_property PACKAGE_PIN C1 [get_ports {an[2]}]
set_property PACKAGE_PIN H1 [get_ports {an[3]}]
set_property PACKAGE_PIN G1 [get_ports {an[4]}]
set_property PACKAGE_PIN F1 [get_ports {an[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg1[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg1[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg1[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg1[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg1[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg1[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg1[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg1[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg2[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg2[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg2[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg2[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg2[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg2[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg2[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg2[0]}]
set_property PACKAGE_PIN B4 [get_ports {seg1[0]}]
set_property PACKAGE_PIN A4 [get_ports {seg1[1]}]
set_property PACKAGE_PIN A3 [get_ports {seg1[2]}]
set_property PACKAGE_PIN B1 [get_ports {seg1[3]}]
set_property PACKAGE_PIN A1 [get_ports {seg1[4]}]

```
set_property PACKAGE_PIN B3 [get_ports {seg1[5]}]
set_property PACKAGE_PIN B2 [get_ports {seg1[6]}]
set_property PACKAGE_PIN D5 [get_ports {seg1[7]}]
set_property PACKAGE_PIN D4 [get_ports {seg2[0]}]
set_property PACKAGE_PIN E3 [get_ports {seg2[1]}]
set_property PACKAGE_PIN D3 [get_ports {seg2[2]}]
set_property PACKAGE_PIN F4 [get_ports {seg2[3]}]
set_property PACKAGE_PIN F3 [get_ports {seg2[4]}]
set_property PACKAGE_PIN E2 [get_ports {seg2[5]}]
set_property PACKAGE_PIN D2 [get_ports {seg2[6]}]
set_property PACKAGE_PIN H2 [get_ports {seg2[7]}]
```

4.板级验证结果



六、实验结果及分析（包括图表、计算、结论与仿真结论）

1. 实验结果分析：

通过 vivado 软件设计电子秒表的几大核心模块，并通过一个顶层文件来连接，从仿真波形图中可以看到最开始复位功能成功运行，多运行一段时间之后，能够成功实现计数，并且完成板级实验结果的验证。

2. 思考题

1、 按键消抖模块中，如何同时对几个按键进行消抖？如何用一个时钟的高电平来表示按键按下的状态？

答：

（1）并行消抖逻辑：为每个按键独立设计一组消抖电路（如计数器或状态机），确保每个按键的抖动信号被单独处理；同步采样：使用同一系统时钟对所有按键输入进行同步化（两级寄存器同步），消除亚稳态；状态检测：通过计数器判断按键状态是否稳定（例如，持续 20ms 高电平/低电平视为稳定），独立判断每个按键的状态；优

化资源：若按键数量较多，可复用计数器逻辑（如分时轮询按键状态）。

（2）在消抖完成后，检测按键信号的上升沿（按下）或下降沿（释放），生成一个时钟周期的高电平脉冲。通过高电平的时间来判断。

2、 数码管驱动模块中， 如何驱动六位数码管？ 如何控制各位数码管小数点的亮灭？

答：

（1）位选信号：通过循环计数器依次选中每一位数码管（如 6 位计数器，频率约 1kHz）；段选信号：根据当前显示内容（如数字 0-9）输出对应段码（共阴极/阳极编码）。

（2）可以为每位数码管设置小数点使能信号（如 6 位独立标志位），动态扫描时根据当前位的小数点状态更新段码。

3、 如何通过例化 RAM IP 核来实现数据存储功能？

答：首先进行 IP 核配置，选择对应的 RAM 类型（单端口/双端口、块 RAM/分布式 RAM），然后设置数据宽度（如 16 位）和深度（如 1024，存储秒表历史记录），并配置读写时序（同步/异步读、写优先模式）。然后通过

以下代码例化： `blk_mem_gen_0 your_ram_instance (`

```
    .clka(clk),      // 时钟
    .ena(ena),       // 使能信号
    .wea(wea),       // 写使能 (1:写, 0:读)
    .addra(addr),    // 地址线
    .dina(data_in),  // 输入数据
    .douta(data_out) // 输出数据 );
```

指导教师签名：

2025 年 月 日